



CODE SECURITY ASSESSMENT

KEYFlexibleStake

Overview

Project Summary

- Name: KEYFlexibleStake
- Platform: BNB Smart Chain
- Language: Solidity
- Address: [0x59CbB5C4f563296462fdcaF63913268106BF0439](#)
- Audit Range: See [Appendix - 1](#)

Project Dashboard

Application Summary

Name	KEYFlexibleStake
Version	v2
Type	Solidity
Dates	Sep 21 2026
Logs	Sep 21 2026; Sep 21 2026

Vulnerability Summary

Total High-Severity issues	0
Total Medium-Severity issues	0
Total Low-Severity issues	1
Total informational issues	1
Total	2

Contact

E-mail: support@salusec.io

Risk Level Description

High Risk	The issue puts a large number of users' sensitive information at risk, or is reasonably likely to lead to catastrophic impact for clients' reputations or serious financial implications for clients and users.
Medium Risk	The issue puts a subset of users' sensitive information at risk, would be detrimental to the client's reputation if exploited, or is reasonably likely to lead to a moderate financial impact.
Low Risk	The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
Informational	The issue does not pose an immediate risk, but is relevant to security best practices or defense in depth.

Content

Introduction	4
1.1 About SALUS	4
1.2 Audit Breakdown	4
1.3 Disclaimer	4
Findings	5
2.1 Summary of Findings	5
2.2 Notable Findings	6
1. Use OpenZeppelin's ECDSA Library Instead of EVM's `ecrecover`	6
2.3 Informational Findings	7
2. Missing Two-Step Ownership Transfer	7
Appendix	8
Appendix 1 - Files in Scope	8

Introduction

1.1 About SALUS

At Salus Security, we are in the business of trust.

We are dedicated to tackling the toughest security challenges facing the industry today. By building foundational trust in technology and infrastructure through security, we help clients to lead their respective industries and unlock their full Web3 potential.

Our team of security experts employ industry-leading proof-of-concept (PoC) methodology for demonstrating smart contract vulnerabilities, coupled with advanced red teaming capabilities and a stereoscopic vulnerability detection service, to deliver comprehensive security assessments that allow clients to stay ahead of the curve.

In addition to smart contract audits and red teaming, our Rapid Detection Service for smart contracts aims to make security accessible to all. This high calibre, yet cost-efficient, security tool has been designed to support a wide range of business needs including investment due diligence, security and code quality assessments, and code optimisation.

We are reachable on Telegram (<https://t.me/salusec>), Twitter (https://twitter.com/salus_sec), or Email (support@salusec.io).

1.2 Audit Breakdown

The objective was to evaluate the repository for security-related issues, code quality, and adherence to specifications and best practices. Possible issues we looked for included (but are not limited to):

- Risky external calls
- Integer overflow/underflow
- Transaction-ordering dependence
- Timestamp dependence
- Access control
- Call stack limits and mishandled exceptions
- Number rounding errors
- Centralization of power
- Logical oversights and denial of service
- Business logic specification
- Code clones, functionality duplication

1.3 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release and does not give any warranties on finding all possible security issues with the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues.

Findings

2.1 Summary of Findings

ID	Title	Severity	Category	Status
1	Use OpenZeppelin's ECDSA Library Instead of EVM's `ecrecover`	Low	Cryptography	Acknowledged
2	Missing Two-Step Ownership Transfer	Informational	Business Logic	Acknowledged

2.2 Notable Findings

Significant flaws that impact system confidentiality, integrity, or availability are listed below.

1. Use OpenZeppelin's ECDSA Library Instead of EVM's ecrecover	
Severity: Low	Category: Cryptography
Target: - KEYFlexibleStake.sol	

Description

The `ecrecovery()` function uses EVM's `ecrecover()` to verify signatures. However, `ecrecover()` does not itself reject malleable signatures. In particular, for a valid signature (r, s, v), another valid signature for the same signer and digest can be derived by replacing s with `secp256k1n - s` and flipping v between 27 and 28.

The current implementation verifies the signature length and v value, but does not require s to be in the lower half of the secp256k1 curve order. While no attack vector has been identified in the current implementation, enforcing canonical signatures follows the highest security standards and avoids treating two different signature byte strings as separate valid approvals in future integrations.

KEYFlexibleStake.sol:L258-L271

```
function ecrecovery(bytes32 hash, bytes memory sig) private pure returns (address) {
    bytes32 r;
    bytes32 s;
    uint8 v;
    if (sig.length != 65) return address(0);

    assembly {
        r := mload(add(sig, 32))
        s := mload(add(sig, 64))
        v := and(mload(add(sig, 65)), 255)
    }
    if (v < 27) v += 27;
    if (v != 27 && v != 28) return address(0);
    return ecrecover(hash, v, r, s);
}
```

Recommendation

Use OpenZeppelin's `ECDSA` library instead of directly using `ecrecover()`.

Status

This issue has been acknowledged by the team.

2.3 Informational Findings

2. Missing Two-Step Ownership Transfer

Severity: Informational

Category: Business Logic

Target:

- KEYFlexibleStake.sol

Description

The `transferOwnership()` function transfers ownership in one step. Although it rejects the zero address, the transfer takes effect immediately and does not require the proposed owner to accept it.

An incorrect nonzero owner address could permanently prevent the team from updating the signer, configuring supported assets, or withdrawing surplus assets. Existing users can still redeem orders because redemption is not owner-controlled.

Recommendation

Use a two-step ownership transfer process where the current owner nominates a `pendingOwner` and the nominated address must explicitly accept ownership. Consider using OpenZeppelin's `Ownable2Step` implementation.

Status

This issue has been acknowledged by the team.

Appendix

Appendix 1 - Files in Scope

This audit covered the following deployed contract at

[0x59CbB5C4f563296462fdcaF63913268106BF0439](https://bscscan.com/address/0x59CbB5C4f563296462fdcaF63913268106BF0439) on BNB Smart Chain:

File	SHA-1 hash
KEYFlexibleStake.sol	FA258495616AE7B912DC4D862EE9A0BF66FF5593